The Relay — Resilient Agentic RAG Benchmark & Systems Evaluation

Aayush Shelar (shelaraayush535@gmail.com) | Target Provider: Groq Free Tier (openai/gpt-oss-20b) & Calibrated FastAPI Stand-In

1 What I built and why

Architecture: I built a 4-step asynchronous Python pipeline (`pipeline.py`) answering exclusively from arXiv 1706.03762v7, 2410.00037v1, and 2507.07955v2: (1) **Query & Paper Router** (deterministic keyword pre-router with a 45-token LLM fallback), (2) **Local BM25 Retrieval Tool** with a stateful section-number successor validator and glyph-density scrubber, (3) **Streaming Grounded Drafter** enforcing strict `[paper_id, p.X]` citations or an explicit `NOT_IN_PAPERS` refusal token, and (4) **Hybrid Critic & Verification Loop** combining a 0-token regex page-citation gate with a targeted LLM grounding check and a 1-turn repair loop. **The One Decision That Shaped It Most:** Choosing *Dual-Stage Buffered Verification* over raw unverified streaming. Because the company never wants to send a wrong or embarrassing answer just to stay fast, tokens streamed from Groq are held in an internal buffer until Step 4 verifies that all citations exist in the retrieved pages and numbers are grounded. This deliberately increases user-visible median TTFT from approx. 420–814 ms (raw draft TTFT) to approx. 980–1,376 ms (verified release TTFT), trading approx. 560–950 ms of initial wait for 100% refusal accuracy on unanswerable queries.

2 The real service and my stand-in

Real Service Limits: Probing Groq's free `on_demand` tier revealed that `llama-3.1-8b-instant` returned HTTP 404 (retired); active free-tier traffic runs on `openai/gpt-oss-20b`. Headers exposed `x-ratelimit-limit-tokens: 8000` (TPM, refilling via a continuous leaky bucket at 133.3 tokens/s) and `x-ratelimit-limit-requests: 1000`. By tracking `x-ratelimit-reset-requests` across sequential probes (7m12s = 432s after 5 requests, or 86.4s/request), I proved the 1,000 limit is **Requests Per Day (RPD)** (1,000 x 86.4s = 86,400s = 24h), while RPM and TPD are masked by the 8,000 TPM bottleneck. **Stand-In Calibration & Differences:** `standin_server.py` is a FastAPI SSE server calibrated to Groq's empirical TTFT distribution, enforcing the 8,000 TPM (133.3 tok/s) and 1,000 RPD leaky buckets, emitting exact Groq 429 JSON payloads + `retry-after` headers, adding linear contention (+180 ms TTFT and +4 ms/token per concurrent request), 1.5% HTTP 500 faults, 2.5% tail-latency spikes (+1.5–2.5s), and streaming cached real Groq responses (`real_response_bank.json`). **Where it differs most:** It models contention linearly rather than simulating real LPU dynamic batching step-functions, lacks prefix KV-caching on repeated system prompts, and uses fixed reasoning-token overhead rather than input-dependent chain-of-thought length.

Limit	Value you found	How you found it	Confidence
Requests per minute	Not observed (masked by TPM)	12-call burst hit 8,000 TPM after 6 calls before RPM triggered	Low
Tokens per minute	8,000 TPM	Header x-ratelimit-limit-tokens: 8000 & 429 body (includes reasoning tokens)	High
Requests per day	1,000 RPD	Header x-ratelimit-limit-requests: 1000 & reset timer +86.4s/call (1/1000 of 24h)	High
Tokens per day	Not observed	Not exposed in HTTP headers; daily token cap not reached during real runs	Low
Requests in flight at once	>= 6 concurrent (no cap seen)	6 of 12 parallel burst calls succeeded simultaneously; rest hit 429 TPM	Medium
What it returns when a limit is hit	HTTP 429 JSON + retry-after	Burst probe (Requested 1622, Remaining 0 -> code: rate_limit_exceeded)	High
Time until it recovers	12.165s (retry-after: 13) for 1,622 tok	Leaky-bucket refill at 133.3 tok/s (8,000/60s); verified by waiting retry-after	High

3 Results

Real vs. Stand-In Runs: All 17 calibration rows (`real_calibration`) and 49 of 54 retrieval evaluation rows in Table 3 (`retrieval_eval`) ran on the **real Groq service** (`source=real`, totaling 66 real calls; the final 5 unanswerable queries of `512w_top5_rerank3` ran on `stand-in`). The 5 load scenarios in Tables 2a/2b (112 requests total) ran against the calibrated **stand-in** (`source=stand-in`). One `results.csv` row equals one end-to-end question; `Peak queue length` is derived from interval overlap of `start_time` and `total_ms`. For Table 2a, latency percentiles are computed over completed answers (`status=ok`) where a first token was emitted, except `daily_budget_exhausted` where 100% fast-fail at 5.3 / 10.5 / 15.5 ms. In Table 2b, `Tokens per answer` and `Cost per 1,000 answers` ($0.10/1M tokens) are computed across all offered requests in each scenario (accounting for tokens consumed prior to 429 termination). **Two Numbers We Could Not Have Guessed Without Running It:** (1) `openai/gpt-oss-20b` streams 4–25 hidden reasoning tokens *before* the first visible answer token, inflating true user TTFT approx. 2.3x over raw first-chunk arrival while quietly burning the 8,000 TPM budget; (2) when Step 4 triggers a single Critic repair loop on `512w_top5_norerank` (e.g., `eval_512w_q05`), resending the multi-turn history balloons prompt consumption to **7,169 input tokens** (89.6% of the entire 8,000 TPM limit in one question!).

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
steady	stand-in	12	274.6	2.6	2077.1 / 4524.1 / 5136.0	2211.4 / 5364.4 / 5389.8
burst	stand-in	25	46.2	32.4	12634.4 / 39170.3 / 44285.0	13318.0 / 39337.3 / 44403.8
sustained_overload	stand-in	40	87.1	27.5	11263.0 / 38606.1 / 43611.3	11527.1 / 39287.1 / 44294.1
per_minute_limit_halved	stand-in	20	439.2	2.7	15920.4 / 36635.9 / 42557.7	16654.5 / 36742.2 / 42664.8
daily_budget_exhausted	stand-in	15	1.5	602.8	not measured	5.3 / 10.5 / 15.5

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
steady	100.0%	0.0%	0.0%	0.42	1	3225	$0.32
burst	20.0%	80.0%	0.0%	2.04	24	549	$0.05
sustained_overload	25.0%	70.0%	5.0%	1.95	13	482	$0.05
per_minute_limit_halved	90.0%	0.0%	10.0%	1.45	1	1819	$0.18
daily_budget_exhausted	0.0%	0.0%	100.0%	0.00	1	0	$0.00

4 Where the time goes

Single Slow Request Anatomy (`eval_512w_q05` on real Groq, `total_ms` = 3,548.3 ms, `first_token_ms` = 3,203.2 ms, `retries` = 1): Splitting total latency across pipeline stages shows: **Waiting (Queue/Backoff) = 0.0 ms** (bucket had sufficient tokens at start); **Initial Working = 1,115.2 ms** (Step 1 Router + Step 2 BM25 tool took 15.2 ms; Step 3 initial Drafter call took 1,100.0 ms); **Retrying / Critic Repair Loop = 2,088.0 ms** (Step 4a regex gate detected a missing bracketed page citation in the initial draft and triggered the repair loop, consuming an extra 3,560 prompt tokens and 2,088.0 ms of Groq inference); **Final Post-Verification Stream = 345.1 ms**. **Before vs. After First Token Proof:** 90.3% of total time (3,203.2 ms of 3,548.3 ms) occurred *before* the first user-visible token. We know this because step trace timestamps record `first_token_ms` only when Step 4 Critic approves the repaired buffer for release (`t_critic_ok` - `t_start` = 3,203.2 ms), proving the delay came from pre-release verification and repair rather than slow token streaming.

5 When a limit is hit

At the Per-Minute Limit (8,000 TPM): Our pipeline tracks `x-ratelimit-remaining-tokens` from response headers. (1) *Who gets a weaker answer:* When remaining TPM drops below 3,800, adaptive retrieval automatically downgrades incoming queries from `512w_top5_norerank` (approx. 4,800 tokens/ans) to `256w_top3_norerank` (approx. 2,323 tokens/ans) and uses the 0-token deterministic citation gate instead of the LLM Critic. This keeps requests succeeding under tight token budgets (`steady` and `per_minute_limit_halved`), at the cost of lower factual recall (61.5% vs. 76.9%). (2) *Who is delayed vs. dropped:* If a call still hits HTTP 429, in-flight requests enter full-jitter backoff (up to 2 retries, delaying tail completions to 39.2–44.3s in `burst` and `sustained_overload`). Requests exceeding `max_retries=2` are dropped cleanly as `rate_limited` (80.0% in burst, 70.0% in sustained_overload with 5.0% timed out) so they do not starve new arrivals. **At the Per-Day Limit (1,000 RPD):** Retrying a 24-hour window is futile; when `remaining-requests == 0` (or reset > 60s), the circuit breaker trips immediately (5.3 ms p50 fast-fail, 10.5 ms p95, 0 retries) and falls back to local extractive BM25 snippets.

Retrieval: quality versus cost

Across 18 custom evaluation questions (13 answerable, 5 unanswerable traps) run on real Groq, `512w_top5_norerank` was selected as our primary default: it achieved the highest overall accuracy (**55.6% correct+cited [10/13 answerable = 76.9%]** and **27.8% correctly refused [5/5 = 100%]**, with only 16.7% wrong) at 4,990 total tokens/answer. `256w_top3_norerank` cut token cost by 49.4% (2,525 tokens/ans) and reduced p95 TTFT from 10,489.8 ms to 1,820.5 ms, but dropped answer recall to 8/13 (44.4% of 18) because small windows split hyperparameter tables. Surprisingly, `512w_top5_rerank3` performed worst on latency (p95 TTFT 22,942.1 ms due to extra repair loops) and reduced recall from 10/13 to 8/13 while slipping to 4/5 on refusals. When the TPM limit is close (< 3,800 remaining tokens), the pipeline dynamically switches from `512w_top5_norerank` to `256w_top3_norerank` to cut token demand in half.

Setting (piece size, pieces fetched, re-rank yes/no)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
256w, top-3, re-rank: no	18	44.4%	27.8%	27.8%	2525	980.0 / 1820.5	1650.6 / 3872.3

Setting (piece size, pieces fetched, re-rank yes/no)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
512w, top-5, re-rank: no	18	55.6%	16.7%	27.8%	4990	1209.8 / 10489.8	2032.6 / 12223.2
512w, top-5->3, re-rank: yes	18	44.4%	33.3%	22.2%	3642	1375.8 / 22942.1	2419.1 / 23050.4

How this library will mislead a naive retriever

1. Cross-Paper Vocabulary Collision: All three papers heavily share terms like *transformer, attention, tokens, layers,* and *latency*. On **Q01** (*"What are N, d_model, and h in the base Transformer?"*) or **Q12** (*"What is the theoretical and practical latency of the spoken dialogue system?"*), an unfiltered retriever pulls H-Net and Moshi architecture sections alongside Vaswani. Our Step 1 Paper Router prevents this via paper-ID metadata filtering. **2. Heuristic Section-Title Boosting Demoting Numeric Tables:** In `512w_top5_rerank3`, boosting chunks whose section title matches query words promoted prose overview sections (e.g., *Section 3.2.2 Multi-Head Attention*) above dense hyperparameter tables, dropping **Q01** (p.3 Table 3, N=6), **Q05** (Moshi p.3, Helium 7B / 2.1T tokens), and **Q12** (Moshi p.1, 200ms practical latency) during top-5-to-3 pruning. **3. PDF Layout & Near-Miss Trap Contamination:** Two-column PDF extraction splits section numbers across lines (`3.1\nEncoder`), turns table cells (`54.3 MPT`) into fake headings, and injects vertical diagram labels (`n d o f t o k e s` on Dynamic Chunking p.3, polluting **Q10**). On near-miss unanswerable traps like **Q15** (*"How does RoPE compare against FlashAttention-2 in Attention Is All You Need?"*), BM25 strongly matches *Attention* and *positional encoding*, misleading a naive drafter into guessing without our `NOT_IN_PAPERS` critic gate.

6 Retries

Policy: We retry HTTP 429 and transient 5xx responses up to `max_retries = 2` using Full-Jitter Backoff anchored on the provider's header: `t_sleep = float(retry_after) + Uniform(0.5, 2.5)`, while never retrying daily budget exhaustion (RPD=0). **One Case Where Retrying Made Things Worse:** During our 12-request real burst probe (and replicated in the 25-request stand-in burst), Groq returned `retry-after: 13` simultaneously to every rate-limited caller. When honoring `retry-after` without jitter, all throttled requests woke up at the exact same millisecond 13.0s later. Because the leaky bucket had only refilled 1,733 tokens (13s * 133.3 tok/s)—barely enough for one call—the synchronized retry wave immediately collided into a secondary 429 thundering herd, burning retry budgets and inflating tail latency to 39.0s. Adding randomized jitter desynchronized wake-ups across the refill slope.

7 What the person sees

At 3:00 PM when the daily budget is exhausted (`daily_budget_exhausted`), the user does **not** wait behind a doomed retry timer. Within 5.3 ms (p50; 10.5 ms p95), the assistant returns an explicit, honest fallback message containing deterministic local BM25 citations so the engineer is still unblocked: *"[DAILY_AI_BUDGET_EXHAUSTED] Our daily language-model verification quota has been reached for today (resets at 00:00 UTC). Rather than guessing without verification, AI synthesis is paused. Here are the top exact passages retrieved locally from the arXiv library for your query: 1) [moshi_2410.00037v1, p.2, Section 1 Introduction]: '...theoretical latency of 160ms and practical latency of 200ms...'"*

8 What I'm unsure about, what I left out, what I learned

Least Certain Assumption & What Breaks: Assuming Groq's 8,000 TPM bucket refills smoothly every millisecond rather than in discrete sub-second windows, and assuming our stand-in's linear contention (+180ms/request) matches LPU hardware batch scheduling. If Groq enforces micro-burst windowing or dynamic priority eviction, our 3,800-token degradation threshold could trigger too late under concurrent 512w repair loops. **Two Things Deliberately Left Out:** (1) *Dense Vector / Hybrid Embeddings:* Omitted to keep local retrieval latency at approx. 3 ms with zero external GPU/API dependencies across a 120-page corpus; (2) *Speculative Sentence-Level Streaming:* Omitted because streaming sentence 1 before sentence 2 passes the Critic risks mid-stream retraction if a later claim fails verification. **One Thing New to Me (In My Own Words):** How reasoning models distort streaming telemetry and token budgets. On `gpt-oss-20b`, the stream opens quickly (approx. 380 ms), but the first several chunks are empty role deltas and internal reasoning tokens (`delta.reasoning`) before any `delta.content` appears. Measuring socket-first-chunk instead of content-first-token understates user wait by more than half, and those unseen reasoning tokens silently drain the per-minute token bucket.

Your feedback on Vaani

Visiting `https://www.vaani.dev/`, **what stood out** is the restraint and clarity of the positioning—focusing directly on natural conversational turn-taking and production voice reliability in Indian languages rather than generic AI buzzwords, backed by an immediately playable live audio demo. **What confused me** is the absence of technical transparency for systems engineers on the landing page: there are no published TTFT / voice-to-voice p50/p95 latency figures, interruption-handling benchmarks, or architectural docs explaining how tool calls avoid dead air. **One thing I would change:** Add a live *"Latency & Telemetry HUD"* toggle right inside the web voice demo showing real-time VAD, ASR, LLM TTFT, and TTS audio-frame timestamps during a conversation—turning the demo itself into proof of engineering speed.

How I worked with the assistant

Three Prompts That Mattered Most: (1) Asking the assistant to translate the brief into concrete systems bottlenecks and cross-checking its output against a second model (Claude) to strip out unverified assumptions; (2) Directing the design of the 4-step pipeline with buffered verification and strict 12-column `results.csv` logging; (3) Auditing the generated PDF structure against Page 2 and Page 3 of the brief to enforce exact heading and table compliance. **One Thing It Got Wrong That I Caught (and How):** The assistant proposed a regex section parser (`SEC_INLINE`) and a numeric grader using word-boundary patterns like `\b8\b` and `\b6\b`, plus initially suggested `llama-3.1-8b-instant`. I caught the model deprecation via live API probing (404 -> `openai/gpt-oss-20b`). I caught the parser bug by writing `test_user_parser.py` and inspecting its output across all 3 papers: it generated approx. 27 fake headings in Moshi and approx. 25 in Dynamic Chunking from numeric table cells (`54.3 MPT`, `0.2474 Watermarked`) and missed headings with parentheses (`2.2 Dynamic Chunking (DC)`). And I caught through unit tests that `\b8\b` falsely graded wrong answers as correct whenever they cited page 8 (`[p.8]`)! **One Decision Where I Overruled It:** The assistant proposed a heuristic bigram + section-title reranker (`512w_top5_rerank3`) expecting it to improve precision. Before wiring it in as default, I wrote an offline recall evaluator (`retrieval_recall.py`) across all 13 answerable questions and proved the reranker actually *dropped* recall from 13/13 to 10/13 by demoting hyperparameter tables—so I overruled it and made `512w_top5_norerank` the primary configuration. **What Is Distinctly Mine:** The successor-validated section state machine (yielding 23/39/35 clean headings), the citation-stripping grader with 8 self-tests, the `delta.reasoning` TTFT fix in the Groq probe, the 86.4s/request mathematical proof that `limit-requests: 1000` is RPD, and the empirical offline recall analysis.